

Program on matrix using CUDA

Name: Satyam Singh

Reg no: 230905256

CSE B2 - 37

Sample Question:

Write a program in CUDA to find transpose of a matrix in parallel.

Ans)

```
#include "cuda_runtime.h"
```

```
#include "device_launch_parameters.h"
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
__global__ void transpose(int *a, int *t) {
```

```
int n = threadIdx.x, m = blockIdx.x, size = blockDim.x, size1 =  
gridDim.x;
```

```
t[n*size+m]=a[m*size+n];
```

```
}
```

```
int main(void) {
```

```
int *a,*t, m, n, i ,j;
```

```
int *d_a,*d_t;
```

```
printf("Enter the value of m: ");scanf("%d",&m);
```

```
printf("Enter the value of n: ");scanf("%d",&n);
```

```
int size = sizeof(int) *m*n;
```

```
a = (int*)malloc(m*n*sizeof(int));
```

```
t = (int*)malloc(m*n*sizeof(int));
```

```
printf("Enter input matrix:\n");
```

```
for(int i=0;i<m*n;i++)
```

```
scanf("%d",&a[i]);
```

```
cudaMalloc((void**)&d_a,size);
```

```
cudaMalloc((void**)&d_t,size);

cudaMemcpy(d_a,a,size,cudaMemcpyHostToDevice);


transpose<<<m,n>>>(d_a,d_t);


cudaMemcpy(t, d_t, size, cudaMemcpyDeviceToHost);


printf("Result vector is:\n");
for(i = 0; i<n; i++) {
for(j = 0; j<m; j++)
printf("%d\t", t[i*m+j]);
printf("\n");
}


getchar();

cudaFree(d_a);

cudaFree(d_t);
```

```
return 0;
```

```
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab8$ ./sample
Enter the value of m: 2
Enter the value of n: 3
Enter input matrix:
1
2
3
4
5
6
Result vector is:
1      4
2      5
3      6
```

Lab Exercises:

Q1) Write a program in CUDA to add two Matrices for the following specifications:

- a. Each row of resultant matrix to be computed by one thread.**
- b. Each column of resultant matrix to be computed by one thread.**
- c. Each element of resultant matrix to be computed by one thread.**

Ans)

```
#include "cuda_runtime.h"
```

```
#include "device_launch_parameters.h"
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
__global__ void matSumRow(int *A, int *B, int *C, int m, int n) {
```

```
int row = threadIdx.x;
```

```
if (row < m)
```

```
for (int col = 0; col < n; col++)
```

```
C[row * n + col] = A[row * n + col] + B[row * n + col];
```

```
}
```

```
__global__ void matSumCol(int *A, int *B, int *C, int m, int n) {
```

```
int col = threadIdx.x;
```

```
if (col < n)
```

```
for (int row = 0; row < m; row++)
```

```
C[row * n + col] = A[row * n + col] + B[row * n + col];  
}
```

```
__global__ void matSumEle(int *A, int *B, int *C, int m, int n) {  
    int row = threadIdx.y;  
    int col = threadIdx.x;  
  
    if (row < m && col < n)  
        C[row * n + col] = A[row * n + col] + B[row * n + col];  
}
```

```
int main(void) {  
    int m, n;  
  
    printf("Enter rows and cols of A (m n): ");  
    scanf("%d %d", &m, &n);  
  
    int *A = (int*)malloc(m * n * sizeof(int));  
    int *B = (int*)malloc(n * n * sizeof(int));
```

```
int *C = (int*)malloc(m * n * sizeof(int));  
printf("Enter matrix A:\n");  
for (int i = 0; i < m * n; i++)  
scanf("%d", &A[i]);  
  
printf("Enter matrix B:\n");  
for (int i = 0; i < n * n; i++)  
scanf("%d", &B[i]);  
int *d_A, *d_B, *d_C;  
  
cudaMalloc((void**)&d_A, m * n * sizeof(int));  
cudaMalloc((void**)&d_B, n * n * sizeof(int));  
cudaMalloc((void**)&d_C, m * n * sizeof(int));  
  
cudaMemcpy(d_A, A, m * n * sizeof(int),  
cudaMemcpyHostToDevice);  
  
cudaMemcpy(d_B, B, n * n * sizeof(int),  
cudaMemcpyHostToDevice);
```

// a) Row-wise

```
matSumRow<<<1, m>>>(d_A, d_B, d_C, m, n);
```

// b) Column-wise

```
// matSumCol<<<1, p>>>(d_A, d_B, d_C, m, n);
```

// c) Element-wise

```
// dim3 threads(p, m);
```

```
// matSumEle<<<1, threads>>>(d_A, d_B, d_C, m, n);
```

```
cudaMemcpy(C, d_C, m * n * sizeof(int),  
cudaMemcpyDeviceToHost);
```

```
printf("\nResult matrix C:\n");
```

```
for (int i = 0; i < m; i++) {
```

```
for (int j = 0; j < n; j++)
```

```
printf("%d ", C[i * n + j]);
```

```
printf("\n");
```

```
}
```



```
cudaFree(d_A);
```

```
cudaFree(d_B);
```

```
cudaFree(d_C);
```

```
free(A);
```

```
free(B);
```

```
free(C);
```

```
return 0;
```

```
}
```

Output:

```

• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab8$ nvcc Q1.cu -o q1
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab8$ ./q1
Enter rows and cols of A (m n): 3 3
Enter matrix A:
1 2 3
4 5 6
7 8 9
Enter matrix B:
9 8 7
6 5 4
3 2 1

Result matrix C:
10 10 10
10 10 10
10 10 10

```

Q2) Write a program in CUDA to multiply two Matrices for the following specifications:

- a. Each row of resultant matrix to be computed by one thread.**
- b. Each column of resultant matrix to be computed by one thread.**
- c. Each element of resultant matrix to be computed by one thread.**

Ans)

```
#include "cuda_runtime.h"
```

```
#include "device_launch_parameters.h"
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
__global__ void matMulRow(int *A, int *B, int *C, int hA, int wA, int wB) {
```

```
int row = threadIdx.x;
```

```
if (row < hA) {
```

```
for (int col = 0; col < wB; col++) {
```

```
int sum = 0;
```

```
for (int k = 0; k < wA; k++)
```

```
sum += A[row * wA + k] * B[k * wB + col];
```

```
C[row * wB + col] = sum;
```

```
}
```

```
}
```

```
}
```

```
__global__ void matMulCol(int *A, int *B, int *C, int hA, int wA, int wB) {
```

```
int col = threadIdx.x;
```

```
if (col < wB) {
```

```
for (int row = 0; row < hA; row++) {
```

```
int sum = 0;
```

```

for (int k = 0; k < wA; k++)
    sum += A[row * wA + k] * B[k * wB + col];
C[row * wB + col] = sum;
}
}
}

```

```

__global__ void matMulEle(int *A, int *B, int *C, int hA, int wA, int wB) {
    int row = threadIdx.y;
    int col = threadIdx.x;

    if (row < hA && col < wB) {
        int sum = 0;
        for (int k = 0; k < wA; k++)
            sum += A[row * wA + k] * B[k * wB + col];
        C[row * wB + col] = sum;
    }
}

```

```

int main(void) {
    int m, n, p;

```

```
printf("Enter rows and cols of A (m n): ");
```

```
scanf("%d %d", &m, &n);
```

```
printf("Enter cols of B (p): ");
```

```
scanf("%d", &p);
```

```
int *A = (int*)malloc(m * n * sizeof(int));
```

```
int *B = (int*)malloc(n * p * sizeof(int));
```

```
int *C = (int*)malloc(m * p * sizeof(int));
```

```
printf("Enter matrix A:\n");
```

```
for (int i = 0; i < m * n; i++)
```

```
scanf("%d", &A[i]);
```

```
printf("Enter matrix B:\n");
```

```
for (int i = 0; i < n * p; i++)
```

```
scanf("%d", &B[i]);
```

```
int *d_A, *d_B, *d_C;
```

```
cudaMalloc((void**)&d_A, m * n * sizeof(int));
```

```
cudaMalloc((void**)&d_B, n * p * sizeof(int));
```

```

cudaMalloc((void**)&d_C, m * p * sizeof(int));

cudaMemcpy(d_A, A, m * n * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, B, n * p * sizeof(int), cudaMemcpyHostToDevice);


// a) Row-wise
matMulRow<<<1, m>>>(d_A, d_B, d_C, m, n, p);


// b) Column-wise
// matMulCol<<<1, p>>>(d_A, d_B, d_C, m, n, p);


// c) Element-wise
// dim3 threads(p, m);
// matMulEle<<<1, threads>>>(d_A, d_B, d_C, m, n, p);


cudaMemcpy(C, d_C, m * p * sizeof(int), cudaMemcpyDeviceToHost);

printf("\nResult matrix C:\n");
for (int i = 0; i < m; i++) {
    for (int j = 0; j < p; j++)

```

```
printf("%d ", C[i * p + j]);
```

```
printf("\n");
```

```
}
```

```
cudaFree(d_A);
```

```
cudaFree(d_B);
```

```
cudaFree(d_C);
```

```
free(A);
```

```
free(B);
```

```
free(C);
```

```
return 0;
```

```
}
```

Output:

```
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab8$ nvcc Q2.cu -o q2
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab8$ ./q2
Enter rows and cols of A (m n): 3 3
Enter cols of B (p): 3
Enter matrix A:
1 2 3
4 5 6
7 8 9
Enter matrix B:
1 2 3
4 5 6
7 8 9

Result matrix C:
30 36 42
66 81 96
102 126 150
```